ALL SYSTEMS GO · SEPT 30, 2026

From fsync to Block I/O

Tracing storage with eBPF

Kritik Sachdeva · IBM

Kritik Sachdeva

Technical Support Professional · IBM

- Work on troubleshooting - the storage / Ceph side
- Today's session: Why I/O latency is your problem day to day
- RHCA Level-11
- Artist by passion



“The disk is slow” — and iostat looks fine

```
Device     r/s     w/s  w_await    f/s  f_await  aqu-sz  %util
nvme0n1    3.0  1180.0     0.31  310.0     0.62    0.85  99.60
```

The device is never idle, and it is never behind either: queue under one, writes in 0.31 ms, flushes in 0.62 ms.

Meanwhile the commit the user is waiting on takes milliseconds.



What no column tells you

- Who waited — iostat counts per device, not per process or per cgroup.
- Where it waited — await starts at issue, so everything before it is off the clock.
- How bad it got — these are averages over the window; the tail is not in them.

The layer we never looked at

We never looked at this layer. For years the disk was obviously the slow part, so “the disk is slow” was a good enough answer.

Compute is now the part we keep adding, and storage is what holds it up — in training runs GPU utilisation often sits below 50%, waiting on data.

So the question I care about is a simple one: where is my I/O actually waiting?



Slowness can be at any layer



The application waits for the whole span. Most dashboards can only show you the right-hand sliver of it.

What eBPF gives you

- A program the kernel verifies and runs on an event. No module, no reboot.
- A timestamp and typed arguments wherever you attach it.
- Counts and histograms kept in kernel maps — cheap enough to leave running.

Where it attaches

`tracepoint`	Static events the kernel already exposes — block:block_rq_issue. Stable across kernels and the cheapest of the three — start here.
`fentry / fexit`	Entry and return of a kernel function, through a BPF trampoline. fexit gives arguments and return value together. Needs kernel 5.5+ and BTF.
`kprobe / kretprobe`	Dynamic — attach at any instruction in a function, not just its entry. Works without BTF; a breakpoint trap, so it costs more and leans on internals.

Demo 1 — cold run, warm run

1. Drop the page cache

   ```
   sync; echo 3 > /proc/sys/vm/drop_caches
   ```

2. Cold run — reads hit the disk

   ```
   dd if=/data/testfile of=/dev/null
   ```

3. Same read again — block layer silent

   ```
   biosnoop · cachestat · blocklat-raw.bt
   ```

Lab Demo

WATCH THE BLOCK TRACE GO QUIET ON THE SECOND RUN

Demo 2 — throttled on purpose

1. Cap the cgroup's IOPS

   ```
   echo "259:0 riops=200" > io.max
   ```

2. Same dd, inside that cgroup

   ```
   dd if=/dev/zero of=/data/out
   ```

3. Device still fast but the wait moved up

   ```
   biolatency -m · blocklat-cgroup.bt
   ```

Lab Demo

WATCH THE HISTOGRAM SHIFT WHILE THE DEVICE STAYS FAST

“Trace the path, correlate the wait, then go look at the device.”

Thank you — questions, and the scripts are yours.

Who waited

This process, this cgroup — not “the host”.

Where it waited

Page cache, journal, block queue, cgroup throttle.

How long

Per layer and per request, not one device average.

Kritik Sachdeva | IBM · Linkedln: @kritiksachdeva · Github: @skritik098